BIZM: Bounded-Isolation Zero-Trust Microsegmentation with Safety-Constrained LLM Agents

Hanh P. Du, Hung K. Luu, Minh T. Nguyen, Hoa N. Nguyen*

Department of Information Systems, VNU University of Engineering and Technology, Hanoi, 100000, Vietnam

Abstract

East–west traffic dominates modern data centers, making lateral movement a critical risk after workload compromise. Zero Trust microsegmentation reduces this risk through least-privilege service-to-service policies, but deployed policies are often static and provide limited runtime response when a compromised workload abuses an otherwise permitted path. IDS and SOAR workflows can raise alerts or execute predefined playbooks, yet fixed playbooks are brittle under unknown signatures and adversarial alert content, and hand-written response rules drift from the architecture they encode. This paper presents **BIZM** (**B**ounded-**I**solation **Z**ero-Trust **M**icrosegmentation), a safety-constrained LLM framework that converts east–west IDS alerts into bounded, auditable isolation actions. BIZM uses the LLM at two speeds. Ahead of time, the LLM compiles the data-center knowledge base into a response program that is verified on every point of the finite alert domain and answers covered alerts without a model call. At runtime, the LLM reasons over the alerts the program does not cover. On both paths, deterministic validation and command-level Zero Trust enforcement admit only source-faithful, provenance-marked, time-bounded, DROP-only actions authorized under a least-privilege role. We implement BIZM on an SDN-based spine–leaf testbed with SONiC switches and Suricata monitoring. Over the 14,970 graded points of the alert domain, the compiled program agrees with the knowledge base on every decision, whereas a hand-written context rule reaches 96.1%. After four knowledge-base changes, recompilation keeps 99.97–100% agreement while the unchanged rule falls to 92.0–96.3%. On the live testbed, BIZM contains all 40 live threats without a false positive, reduces alert-to-decision latency from 3.1 to 0.6 seconds, keeps deciding during a model-provider outage, and produces no unsafe enforcement under adversarial alerts, whereas the same LLM without safety controls violates safety in every adversarial case. These results indicate that LLM-assisted microsegmentation response is feasible on a controlled testbed when the LLM compiles verified response logic from the architecture and its runtime authority is bounded by deterministic safety invariants.

Keywords: Zero Trust, Microsegmentation, East-West Threats, Lateral Movement, LLM Agent, Automated Threat Response, Software-Defined Networking, Network Security

1. Introduction

Modern data centers are increasingly dominated by east–west communication among internal services rather than by north–south traffic crossing the enterprise perimeter. This shift is driven by cloud computing, virtualization, microservices, and disaggregated service architectures, where a single user-facing request may trigger multiple interactions across web, application, database, storage, and management services [1]. Once one workload is compromised, the internal service mesh becomes the primary surface for reconnaissance, privilege escalation, lateral movement, command-and-control communication, and data exfiltration. Perimeter-oriented defenses are therefore insufficient because they provide limited visibility and control over service-to-service communication inside the data center.

Zero Trust Architecture (ZTA) reduces this risk by removing implicit trust from the internal network and enforcing continuous verification, least-privilege access, and resource-centric policy control [2, 3]. Within ZTA, microsegmentation partitions workloads into fine-grained security zones and permits only explicitly authorized inter-zone communication. Existing studies and guidance show that microsegmentation can reduce attack-surface exposure and improve internal network robustness [4, 5, 6, 7, 8]. However, most deployed policies remain static: they are derived from baseline traffic or design-time dependency analysis and enforced until an operator revises them. Static microsegmentation can block denied zone pairs, but it does not decide how to contain a compromised workload that abuses an otherwise permitted path.

East–west detection has advanced in parallel. Graph-

*Corresponding author

Email addresses: hanhdp@vnu.edu.vn (Hanh P. Du), 22024546@vnu.edu.vn (Hung K. Luu), 23020628@vnu.edu.vn (Minh T. Nguyen), hoa.nguyen@vnu.edu.vn (Hoa N. Nguyen)

based and temporal systems such as NetVigil and Euler detect anomalous internal communication and lateral movement at scale [9, 10]. Signature-based IDS engines such as Suricata and Snort provide practical, explainable, and low-latency alerts for known threats [11], while AI-powered IDS research has improved traffic sensing, flow-level analysis, data augmentation, ensemble inference, and adversarial robustness [12, 13]. Yet detection alone does not close the response loop. In operational settings, IDS alerts are commonly triaged by analysts or mapped to actions through SOAR playbooks. Manual response may be too slow for lateral-movement containment, whereas fixed playbooks generalize poorly to unknown signatures, adversarial alert content, and context-dependent permitted-path abuse.

This paper focuses on the resulting network-response problem: given an east–west alert, how can a system decide whether temporary isolation is warranted, determine the correct scope, and enforce the action without turning the decision component into an over-privileged network operator? A correct response depends on segmentation policy, topology, asset role, baseline communication, alert semantics, incident context, and the operational risk of blocking protected services. Deterministic rules can encode this context, but they are written for one version of the architecture and silently diverge from it as signatures, zones, and documented responses change. Large Language Models (LLMs) are useful because they can combine heterogeneous context, follow structured schemas, invoke tools, and generate explanations [14, 15, 16, 17, 18, 19]. However, placing an LLM directly on the enforcement path is unsafe: it may hallucinate, misread adversarial alert text, emit malformed outputs, or target the wrong asset [20, 21]. The challenge is therefore threefold: the response logic must handle permitted-path abuse, automated isolation must remain safe under uncertain or adversarial inputs, and the authority to change the fabric must remain deterministic, least-privileged, auditable, and fail-closed.

This paper presents **Bounded-Isolation Zero-Trust Microsegmentation (BIZM)**, a safety-constrained LLM framework for adaptive east–west response. BIZM converts IDS alerts into bounded isolation decisions rather than direct network commands. Its design follows a separation principle: contextual reasoning and enforcement authority must remain separate. The LLM reasons over alert semantics, topology, segmentation policy, asset roles, baseline communication, and incident history, but it can only propose a candidate bounded-isolation decision. Deterministic validation and command-level Zero Trust enforcement decide whether that candidate is admissible before any rule reaches the fabric.

BIZM uses the LLM at two speeds. Most alerts carry a catalogued signature on a known zone pair, and their correct response follows from the architecture knowledge base. For these alerts, BIZM moves reasoning ahead of time: the LLM compiles the knowledge base into a response program, and a verifier checks the safety invariants on every point of the finite alert domain before the program may act. The verified program answers covered alerts in milliseconds without a model call and abstains on the rest, which the LLM then reasons over at runtime. When the knowledge base changes, the program is recompiled rather than rewritten by hand.

The core mechanism is *bounded isolation*. BIZM restricts automated actions to source-faithful, provenance-marked, time-bounded DROP rules against the offending source. Each candidate decision is checked against fail-closed controls, including schema validation, prompt-injection sanitization, topology checking, protected-asset exclusion, off-target rejection, blast-radius limits, severity and confidence gates, and circuit breaking. The enforcement layer authenticates each request, binds it to a restricted agent identity, authorizes the operation at command granularity, and rejects unsupported or over-privileged actions. Thus, safety is not delegated to the LLM; it is enforced by explicit invariants and least-privilege control.

BIZM complements rather than replaces east–west detection. The IDS supplies the suspicious signal, while BIZM decides whether that signal warrants temporary source isolation under topology, policy, baseline, and asset-safety constraints. This distinction is essential for permitted-path abuse, where an application host may legitimately access a database while using the same allowed edge for destructive queries or bulk extraction. The novelty of BIZM lies in coupling LLM-based contextual reasoning with deterministic network authority under a bounded action contract: the model contributes interpretation, whereas the architecture controls admissibility, target, scope, privilege, and execution before any rule reaches the fabric.

The main contributions of this paper are as follows:

- We formulate **bounded isolation** as a safety-constrained response problem for adaptive Zero Trust microsegmentation, defining a minimal action space and admissibility invariants for action restriction, target fidelity, protected-asset exclusion, bounded scope, confidence gating, and provenance-bound authorization, and we show that the invariants can be verified exhaustively over a finite alert domain.

- We introduce **LLM response-policy compilation**: the LLM translates the architecture knowledge base into a program in a target-free response language, and counterexample-guided repair, self-review, and a majority ensemble produce a program that is verified on every input and activated only for the knowledge base it was compiled from.

- We design **BIZM**, a two-speed architecture in which the compiled program answers covered alerts and a graph-grounded LLM agent reasons over the residual alerts, with both paths bound by the same fail-closed validation and least-privilege enforcement contract on the SONiC-based spine–leaf fabric.

- We implement and evaluate BIZM on an SDN-based spine–leaf testbed with Suricata monitoring. The evaluation covers exhaustive agreement with the knowledge base, recompilation under knowledge-base change, live closed-loop containment, adversarial alert manipulation, protected-asset invariants, and unknown signatures, against signature-keyed, hand-written contextual, and guardrail-free LLM baselines.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 formulates the bounded-isolation problem. Section 4 presents the BIZM architecture and workflow. Section 5 describes the implementation and experimental design. Section 6 reports and discusses the evaluation results. Section 7 concludes the paper.

2. Related Work

We review prior work along four lines: Zero Trust microsegmentation and east–west threat detection, LLM-assisted network operations, automated threat response and programmable mitigation, and least-privilege enforcement for network automation. We then summarize the gap addressed by BIZM.

2.1. Zero Trust Microsegmentation and East–West Threat Detection

Microsegmentation is a core mechanism for realizing Zero Trust in data centers. It partitions workloads into fine-grained security zones and restricts east–west communication to explicitly authorized service dependencies. Beyond NIST SP 800-207, recent surveys and guidance have refined Zero Trust principles for cloud-native, distributed, and multi-location environments [3, 5, 6, 7]. NIST SP 800-204A also provides microservices-oriented guidance for workload-level segmentation and service-to-service control [8]. In most deployments, however, microsegmentation policies are derived from baseline traffic, role inference, or dependency analysis and enforced until an operator revises them. They reduce the attack surface, but they remain largely static: they can block denied zone pairs, yet they do not decide how to contain a compromised workload that abuses an otherwise permitted path.

Recent studies have improved segmentation policy automation. Game of Zones derives security zones from high-level intents using Answer Set Programming [23]. ZT-SDN learns Zero Trust access-control rules from graph-modeled network transactions and enforces them through an SDN controller [24]. Mani et al. infer endpoint roles from flow logs and domain knowledge for public-cloud microsegmentation [22], while Hari et al. explore adaptive Zero Trust microsegmentation through a Bayesian trust score [31]. These works show that segmentation can be learned, synthesized, or adapted, but their primary focus is policy construction or policy adjustment rather than per-alert runtime containment under explicit safety invariants.

A parallel line of work focuses on detecting east–west threats. NetVigil extracts graph features from low-cost flow logs and applies domain-specific graph neural networks for data-center anomaly detection [9]. Euler models host activity as temporal graphs and detects lateral movement through anomalous link prediction [10]. Self-supervised sequence models have also been used to detect red-team lateral movement from authentication logs [32]. In addition, AI-powered IDS research has improved traffic sensing, flow-level analysis, data augmentation, ensemble inference, and adversarial robustness [33, 34, 12]. These systems provide valuable signals for adaptive defense, but they stop at detection or alert prioritization. They do not determine whether a specific source should be isolated, compute a bounded enforcement scope, or provide a least-privilege contract for applying a response to the fabric.

2.2. LLM-Assisted Network Operations

LLM-assisted network operations have attracted attention because language models can translate intent, retrieve technical context, generate structured outputs, and assist with complex configuration tasks. Dzeparoska et al. use LLMs to decompose high-level application-management intents into policy-based execution steps [35]. NetConfEval evaluates whether LLMs can support network configuration and routing-related tasks, showing both promise and fragility in deployable settings [25]. LLNet studies how small language models can translate intents into softwarized network-device instructions [26]. Siino et al. integrate LLMs into network testbeds for automated experimentation and optimization [36]. General agent techniques, including tool use, self-consistency, and self-refinement, further motivate structured reasoning pipelines [17, 18, 19, 20].

These studies bridge high-level intent and low-level network artifacts, but they mainly address configuration generation, intent decomposition, device programming, or experiment automation, often with an operator or test workflow in the loop. BIZM instead places an LLM inside a security response loop, where an unsafe decision may disrupt production services. The model can therefore only propose a bounded isolation decision; deterministic validation and identity-aware command authorization decide whether the proposed action can reach the fabric. This distinction is central to security automation: correctness of generated text is not enough when the output may become a live network operation.

A second line pairs LLM generation with verification. Clarify synthesizes ACL and route-map stanzas with an LLM, checks them with Batfish, and asks the operator to disambiguate overlapping intents [27]. Yamamoto and Katsurai synthesize intrusion-detection specifications with an LLM and refine them against benign traffic in a counterexample-guided loop before compiling them into Suricata rules [37]. These systems verify configuration or detection artifacts. BIZM applies the generate-and-verify principle to the response decision itself: the LLM writes a response program for the whole alert domain, and the

Table 1: Positioning of BIZM among recent east–west security, LLM-networking, and automated-response systems. The comparison focuses on defensive-loop capability rather than absolute performance because the systems target different infrastructures and stages of the response pipeline.

System	Core method	Response decision	Fabric enforcement	Bounded safety	Closed loop
NetVigil [9]	Unsupervised GNN over east–west flow logs	No	No	No	No
Euler [10]	Temporal graph link prediction for lateral movement	No	No	No	No
Role-based microsegmentation [22]	Role inference for cloud microsegmentation policy	No	Design-time	No	No
Game of Zones [23]	Intent-based security-zone synthesis	No	Design-time	No	No
ZT-SDN [24]	Learned Zero Trust rules in SDN	Policy learning	Yes	Partial	Partial
NetConfEval [25] / LLNet [26]	LLM-assisted network configuration	Config generation	No	No	No
Clarify [27]	LLM configuration synthesis with formal verification	Config generation	No	Partial	No
DAERF [28] / P4-Secure [29]	Classifier-driven SDN/P4 mitigation	Coarse mitigation	Yes	No	Yes
Dynamic ZT zoning [30]	Stochastic-game policy adaptation	Policy adaptation	Yes	No	Yes
BIZM	LLM-compiled verified response policy with bounded runtime reasoning	Yes	Yes	Yes	Yes

safety invariants are checked on every input before the program may act.

2.3. Automated Threat Response and Programmable Mitigation

Automated threat response commonly follows three directions: rule-based playbooks, classifier-driven mitigation, and programmable data-plane defense. Rule-based SOAR systems map alert types to predefined actions and are attractive because they are simple and auditable, but they generalize poorly to unknown signatures, ambiguous contexts, and permitted-path abuses that require topology and baseline reasoning. Incident-response guidance also emphasizes that containment actions should be scoped, reversible, and auditable when they may affect business-critical services [38, 39, 40]. Machine-learning systems improve detection and prioritization, including under adversarial evasion and malware-detection settings [13, 41, 42, 43], but they typically output scores, classes, or coarse mitigation signals rather than bounded per-alert isolation decisions.

Several works couple detection and mitigation more tightly in SDN or programmable networks. DAERF integrates a hybrid deep autoencoder and random forest model into an SDN mitigation framework [28]. P4-Secure performs in-band DDoS detection directly in the programmable data plane [29]. P4DDLe extracts packet-level features in programmable data planes to improve intrusion detection while reducing unnecessary control-channel load [44]. The broader programmable-networking literature provides mechanisms for installing and placing rules through OpenFlow, P4, ACL systems, and match-action pipelines [45, 46, 47, 48]. These mechanisms improve how and where enforcement can be realized, but they do not decide whether an automated isolation action is admissible under Zero Trust safety constraints.

Other systems adapt detector or policy logic at runtime. Auto-IDaS uses reinforcement learning to configure IDS models, tasks, and capacities for Intrusion Detection as a Service [49]. Bello and Refaey Hussein propose runtime policy decision and enforcement for Zero Trust security zoning through stochastic games and meta-learning [30]. These approaches are adaptive, but they operate at a different abstraction level: Auto-IDaS optimizes the detection service, while dynamic security zoning adapts policy in mobile-edge settings. BIZM instead evaluates an individual east–west alert against a data-center segmentation baseline and applies only a constrained, auditable, temporary source-isolation rule.

Recent SOC studies combine deterministic and LLM stages. Huynh et al. route alerts through a rule engine and use a local LLM only for advisory analysis, and they conclude that deterministic processing should remain the default for alert categories already covered by rules [50]. SOCpilot verifies LLM-generated response plans against a fixed policy and action catalogue [51], and LLM cascades defer or abstain on queries that a cheaper stage cannot answer [52]. BIZM follows the same division of labour, but the LLM derives the deterministic stage from the knowledge base, so the rule tier is neither written nor maintained by hand.

2.4. Least-Privilege Enforcement for Network Automation

Autonomous network response is unsafe unless the response component is itself constrained. This issue becomes more acute when LLMs are used in security automation, because the model may hallucinate, be manipulated by adversarial alert text, or generate syntactically valid but operationally unsafe actions. Prior work on Zero Trust, SDN security, and intent-based management highlights identity verification, policy enforcement, conflict detection, and protocol-aware control [3, 53]. These mechanisms, however, are usually designed for operator commands, service intents, or static policy deployment, not for bounding the authority of an autonomous response component.

Policy-enforcement and configuration-synthesis systems address related but different problems. I2NSF policy-conflict detection prevents inconsistent security policies before deployment [54]. Automated firewall synthesis and verification generate conflict-free configurations for virtual networks [55]. ACL placement methods optimize where rules should be installed under hardware constraints [56]. These works improve policy correctness, placement, and conflict handling, but they generally assume that a policy or intent has already been accepted as legitimate. They do not ensure that an automatically generated isolation command is within the agent's authority, targets only the alert source, avoids protected assets, and carries a bounded lifetime and provenance mark.

BIZM therefore treats least-privilege enforcement as part of the response architecture. The response component is bound to a restricted role whose admissible operation is limited to creating temporary DROP rules. Requests to create ACCEPT rules, delete unrelated rules, modify baseline configuration, target protected assets, or block an address different from the alert source fail closed. In this way, Zero Trust is instantiated not only as a segmentation principle but also as a safety envelope for LLM-assisted network response.

2.5. Research Gap and Approach Direction

The reviewed literature leaves a common gap: existing systems improve segmentation design, east–west detection, LLM-assisted configuration, programmable mitigation, or policy enforcement, but they do not jointly provide contextual per-alert reasoning, explicit isolation invariants, protected-asset safety, provenance, and command-level least privilege in a single closed response loop, and none derives a verified deterministic response tier from the architecture description. BIZM addresses this gap by framing adaptive microsegmentation response as a bounded-isolation problem in which the LLM compiles verified response logic ahead of time and proposes candidate actions at runtime, while the surrounding architecture controls admissibility, target, scope, and enforcement authority. Because the closest systems operate on different infrastructures and occupy different stages of the defensive loop, a direct numerical comparison would be misleading. Table 1 therefore positions BIZM by defensive-loop capability rather than by absolute performance.

3. Problem Formulation

3.1. System and Network Model

We consider a data-center fabric partitioned into a set of security zones $Z = \{z_1, \ldots, z_n\}$. Each zone groups workloads with a common operational role, such as web, application, database, monitoring, or management. The zones are interconnected through a spine–leaf fabric, where leaf switches serve as the primary enforcement points because they are closest to the workloads. A static microsegmentation policy is represented as

$$P : Z \times Z \to \{\textsc{allow}, \textsc{deny}\},$$

where the default decision is DENY. A zone pair is permitted only if it is explicitly listed in P. The policy is compiled into data-plane rules on the leaf switches, so traffic on denied zone pairs is blocked independently of the response system.

A network flow is modeled as $f = (s, d, \rho, \pi, \phi)$, where s and d are the source and destination addresses, ρ is the transport protocol, π is the destination service port, and ϕ denotes behavioral features such as connection rate, byte volume, session duration, periodicity, and request pattern. Let $z(s)$ and $z(d)$ denote the zones of the source and destination endpoints. A flow is statically permitted if $P(z(s), z(d)) = \textsc{allow}$. Static permission, however, does not imply benign behavior. We therefore define a legitimate communication model

$$B \subseteq \{f \mid P(z(s), z(d)) = \textsc{allow}\},$$

which captures expected communication patterns for permitted zone pairs, including service ports, directionality, traffic volume, and temporal behavior. The model B provides the baseline against which permitted-path traffic is judged.

An east–west IDS observes mirrored inter-zone traffic and emits alerts for policy-relevant violations, signature matches, or suspicious behavioral patterns. An alert is represented as

$$a = (s, d, \rho, \pi, \sigma, \ell, \kappa, \tau, t),$$

where σ is the signature identifier, ℓ is the severity level, κ is the alert category, τ is the textual signature or alert description, and t is the timestamp. The IDS is treated as the source of suspicious evidence. The problem addressed in this paper is not raw traffic detection, but response-level decision making: given an alert, decide whether bounded source isolation is warranted and whether the resulting action can be safely enforced.

3.2. Threat Model

We assume an adversary that has compromised one or more internal workloads and operates over east–west traffic. The adversary may perform reconnaissance, lateral movement, privilege escalation, command-and-control communication, destructive queries, or data exfiltration. We consider two primary classes of malicious behavior.

The first class is a *policy violation*. A compromised workload initiates traffic on a denied zone pair, such as a web host connecting directly to a database. The static policy may already block the violating packets, but the attempt still indicates a potentially compromised source. A response system must therefore decide whether the source itself should be temporarily isolated.

The second class is *permitted-path abuse*. A compromised workload uses an allowed zone pair but deviates from the legitimate communication model B. For example, an application service may be allowed to access a database, but the same path can be abused for high-rate extraction, destructive commands, or abnormal request sequences. Static microsegmentation cannot block such behavior without also disrupting legitimate service traffic, because the zone pair and service port may remain valid. A response decision must therefore account for baseline behavior, asset role, zone relationship, and the operational impact of isolation.

The adversary may also attempt to manipulate the response layer. Alert fields such as τ and κ may contain attacker-influenced text, including prompt-injection instructions that attempt to redirect isolation to a protected management host, an IDS component, or an unrelated workload. The source field may also reflect spoofing, detector error, or incomplete telemetry. We therefore distinguish transport validity from semantic trust. The alert transport is assumed to be syntactically available, but alert content is treated as untrusted input to the decision component. The decision component is also treated as untrusted for enforcement: it may hallucinate, misinterpret context, produce malformed output, or select an unsafe target. No model output is allowed to reach the fabric unless it satisfies deterministic safety checks and least-privilege authorization.

We trust the enforcement layer to implement authentication, authorization, provenance stamping, and command-level rejection correctly. We do not consider compromise of the switch operating system, compromise of the enforcement implementation itself, inspection of encrypted payloads beyond what the IDS can observe, or north–south traffic. These are important deployment concerns, but they are outside the scope of the bounded east–west response problem studied here.

3.3. Bounded-Isolation Decision Problem

Given an alert a, the system produces a response decision d and, when warranted, enforces it as a temporary data-plane rule. A decision is defined as

$$d = \langle \alpha, \theta, \rho, \pi, \delta, c, m \rangle,$$

where α is the response action, θ is the enforcement target, ρ and π identify the protocol and destination port, δ is the rule time-to-live, c is the decision confidence, and m is the MITRE ATT&CK classification [57, 58]. The action space is intentionally minimal: $\alpha \in \{\textsc{drop}, \textsc{none}\}$.

The action DROP creates a temporary isolation rule against the offending source, whereas NONE denotes logging, audit recording, or human referral without automated enforcement.

The objective is to maximize correct bounded containment while limiting closed-loop response latency. Correct bounded containment means issuing an admissible DROP decision for true threats whose sources can be safely isolated, and issuing NONE for benign, ambiguous, low-severity, or unsafe-to-enforce cases. Let $y(a) \in \{\textsc{threat}, \textsc{benign}\}$ denote the ground-truth status of the alert context, and let $\mathcal{A}(a, d)$ be an admissibility predicate that returns true only when d satisfies the bounded-isolation invariants in Section 3.4. The response problem can be written as

$$\begin{aligned} \max_{d} \ & \text{Containment}(a, d) - \lambda \, \text{Latency}(a, d), \\ & \text{s.t. } \mathcal{A}(a, d) = \textsc{true}. \end{aligned} \tag{1}$$

In BIZM, this objective is realized through a structured decision pipeline followed by deterministic fail-closed validation. The optimization form captures the design goal; the implementation enforces the constraint through explicit invariants rather than relying on unconstrained model behavior.

3.4. Bounded-Isolation Invariants

A decision d is admissible only if it satisfies the following invariants.

- **(I1) Action restriction.** The response action must satisfy $\alpha \in \{\textsc{drop}, \textsc{none}\}$. Automated response may create a temporary DROP rule or take no enforcement action. It must not create an ACCEPT rule, delete another entity's rule, modify unrelated configuration, or perform general network-management operations.
- **(I2) Target fidelity.** The enforcement target must satisfy $\theta = s$. The blocked address must be the alert source as a single host address; a prefix that merely contains the source is rejected. This prevents hallucinated, injected, or off-target isolation actions from being enforced.
- **(I3) Protected-asset exclusion.** The target must satisfy $\theta \notin \mathcal{N}$, where $\mathcal{N}$ is an immutable protected set that includes control-plane components, IDS components, zone gateways, management services, and monitoring services. These assets cannot be isolated automatically, even if an alert names them as the source. A rule scoped to a non-protected source may still restrict that source's access to a protected destination, because it drops only connections the source initiates and cannot isolate the destination.

- **(I4) Bounded scope.** Every enforced DROP rule must carry a time-to-live $\delta \in [\delta_{\min}, \delta_{\max}]$ and must satisfy blast-radius limits on the number of rules installed per minute, the total number of active automated rules, and the number of rules per source within a time window.
- **(I5) Severity and confidence gating.** Automatic enforcement is allowed only when the IDS severity satisfies $\ell \in \{\mathrm{P1}, \mathrm{P2}\}$ and the decision confidence satisfies $c \geq \tau_c$. Lower-severity, low-confidence, ambiguous, or incomplete cases are logged, audited, or referred for human review.
- **(I6) Provenance and least-privilege authorization.** Every enforced rule must carry a provenance mark that binds it to the automated response path and must be authorized under a least-privilege role at command granularity. The response component is authorized only to create bounded DROP rules; unauthenticated, over-privileged, or unsupported operations are rejected before reaching the fabric.

Invariants I1, I2, I3, and I6 are architectural invariants: they must hold regardless of the model output. Invariants I4 and I5 are deployment policies that bound operational risk and can be tuned by administrators. Together, these invariants define the bounded action contract exposed to the decision component.

3.5. Safety Argument

The bounded-isolation invariants provide a safety-by-construction guarantee for the most critical unsafe response classes.

Proposition 1. If the validation and enforcement layers are correctly implemented, no rule installed by BIZM can violate I1, I2, I3, or I6, regardless of the output produced by the LLM decision component.

Proof sketch. I1 holds because the enforcement layer accepts only rule-creation requests whose action is DROP; ACCEPT, deletion, and unrelated configuration operations are outside the authorized action set and are rejected. I2 holds because the validator checks that the requested target equals the alert source before provenance stamping or enforcement. I3 holds because the protected set $\mathcal{N}$ is maintained outside the decision component and checked after model output but before rule installation. Any target in $\mathcal{N}$ is rejected regardless of the model rationale. I6 holds because every accepted request must be authenticated, bound to a restricted response role, authorized at command granularity, and stamped with provenance metadata before reaching the fabric. Therefore, malformed, injected, hallucinated, or over-privileged model outputs can only result in rejection or non-enforcement, not in unsafe rule installation.

This proposition does not claim that the model always selects the optimal response. It establishes a narrower property that is operationally more important for safety: even when the model is wrong, its authority remains bounded by deterministic controls. The model may fail to recommend isolation or may produce a low-confidence decision, but it cannot acquire the authority to install an arbitrary or protected-asset-disrupting rule.

The compiled response policy of Section 4.4 admits a stronger property, because its decisions can be checked before any alert arrives.

Proposition 2. Let K be a knowledge-base snapshot, $D(K)$ the finite domain of alert features defined by K, and Π a compiled program that the verifier accepts on every point of $D(K)$. While the deployed knowledge base equals K, every decision issued by Π satisfies I1 to I5 for any alert, including spoofed or internally inconsistent alerts.

Proof sketch. I1 and I2 hold by construction: the program language admits only DROP, LOG_ONLY, and ABSTAIN, and has no target field, so an enforced rule always names the alert source. The feature map is total: it sends a signature outside the catalogue to a single unknown value, an address outside the fabric to an external zone, and a severity outside P1 to P4 to P4, so every alert maps to a point of $D(K)$. The verifier evaluates Π on all points of $D(K)$ and checks I3, I4, and I5 on each decision, so no alert can produce a violating decision. The fast path is enabled only when the digest of the deployed knowledge base equals that of K and Π passes verification again at load time.

3.6. Design Requirements

The system and threat models lead to the following requirements.

- **(R1) Closed-loop bounded containment.** The system must convert a severe east–west alert into an enforced temporary isolation rule without human intervention when the decision is confident and all safety invariants are satisfied.
- **(R2) Contextual decision-making.** The system must reason beyond a fixed signature-to-action map. It must handle permitted-path anomalies, unknown signatures, and benign high-volume communication using topology, policy, baseline behavior, asset roles, and incident context, and it must follow the architecture knowledge base as it changes without hand-maintained decision tables.
- **(R3) Fail-closed safety.** Malformed output, prompt injection, off-target action, protected-asset targets, low-confidence decisions, policy conflicts, and validation errors must result in non-enforcement rather than incorrect enforcement.
- **(R4) Least-privilege enforcement.** The response component must be treated as an untrusted subject and bound to a restricted role whose only automated authority is to create temporary DROP rules within the bounded action contract.

- **(R5) Auditability and provenance.** Every decision must record the alert, retrieved context, candidate decision, validation result, reasoning trace, MITRE mapping, time-to-live, provenance mark, and enforcement outcome.
- **(R6) Substitutability.** The decision model, IDS, and network devices must be replaceable behind stable interfaces. The bounded response logic must not depend on a specific LLM provider, IDS engine, switch operating system, or device-management protocol.
- **(R7) Bounded operational impact.** The architecture must limit the blast radius of automation through time-bounded rules, protected-asset exclusions, per-source and global rate limits, and circuit breaking under repeated enforcement failures.
- **(R8) Predictable decision path.** Alerts whose response is determined by the knowledge base must be decided without a model call, so that decision latency, cost, and availability for known threats do not depend on the model provider.

4. Proposed Method: BIZM Architecture and Bounded-Isolation Workflow

BIZM is a bounded-response architecture for Zero Trust microsegmentation in a SONiC-based data-center fabric. Its goal is to close the loop from east–west alerting to temporary source-side containment without giving the LLM direct authority over the network. The LLM Security Agent may interpret alerts, retrieve context, and propose a bounded isolation action, but every candidate response must pass deterministic validation and identity-aware control-plane mediation before it can become a device-level DROP rule.

BIZM is organized into four planes. The *data plane* forwards workload traffic, enforces static segmentation policy, mirrors selected east–west traffic to the IDS, and installs temporary DROP rules on the SONiC fabric. The *decision plane*, implemented as the LLM Security Agent, answers covered alerts from a compiled response policy and, for the remaining alerts, classifies them, retrieves GraphRAG context, and performs LLM-based reasoning; it then validates candidate decisions and generates policy/YANG-oriented bounded-response requests. The *control plane* mediates enforcement through the IDS Agent, the Secure Framework, and ONAP SDN-C. The *operator layer* provides live visibility into alerts, flows, candidate decisions, validation outcomes, and enforcement status. This separation keeps contextual reasoning, safety validation, and command-level enforcement under distinct control responsibilities.

4.1. Architecture Overview

Figure 1 shows the overall BIZM architecture and the bounded-isolation message flow. The workflow begins when the SONiC spine–leaf fabric mirrors selected east–west traffic to Suricata IDS through SPAN or ERSPAN. Suricata inspects the mirrored stream and emits EVE JSON alerts. These alerts are carried through an SSE-based telemetry path to the IDS Agent and then to the LLM Security Agent. The LLM Security Agent normalizes the alert and evaluates the compiled response policy. If the policy abstains, the agent classifies the alert, retrieves graph-grounded context, and invokes the reasoning engine. It then validates the candidate decision and generates a bounded DROP request when containment is admissible. The request is pushed to the IDS Agent, forwarded to the Secure Framework through REST, authorized under mTLS and RBAC, coordinated by ONAP SDN-C, and finally realized on the SONiC fabric through the southbound management path. Alerts and flow updates are also streamed to the operator dashboard for monitoring and audit.

The numbered flow in Figure 1 contains six steps. First, the fabric mirrors traffic to Suricata IDS. Second, Suricata generates EVE JSON alerts. Third, alerts are delivered to the LLM Security Agent, which answers them from the compiled policy or, when it abstains, through classification, context retrieval, and reasoning. Fourth, if bounded containment is justified, the rule generator pushes a policy request to the IDS Agent. Fifth, the IDS Agent forwards the request to the Secure Framework, where authentication, authorization, and policy mediation are applied. Sixth, the accepted rule is enforced through ONAP SDN-C and the southbound gNMI-based control path. Thus, telemetry flows upward from the data plane to the decision plane, whereas policy commands flow downward only after validation and identity-aware mediation.

The architecture provides two safety barriers. The first is the fail-closed validation path inside the LLM Security Agent, which rejects malformed decisions, low-confidence responses, off-target rules, protected-asset violations, TTL violations, and blast-radius violations before any control-plane request is generated. The second is the identity-aware enforcement path in the control plane, which restricts accepted requests to authorized DROP-only operations and installs them through a mutually authenticated management channel. The LLM may therefore contribute contextual reasoning, but it cannot acquire arbitrary network-management authority.

4.2. Data Plane and Monitoring

The data plane is implemented as a SONiC-based spine–leaf fabric. Leaf switches are the closest enforcement points to workloads and therefore serve as the locations where temporary source DROP rules are installed. BIZM assumes a deny-by-default segmentation baseline: inter-zone communication is allowed only when explicitly permitted by the policy matrix, and all other east–west paths are blocked by default. Runtime containment is applied at the source-side leaf switch, which limits the scope of the response and avoids fabric-wide disruption.

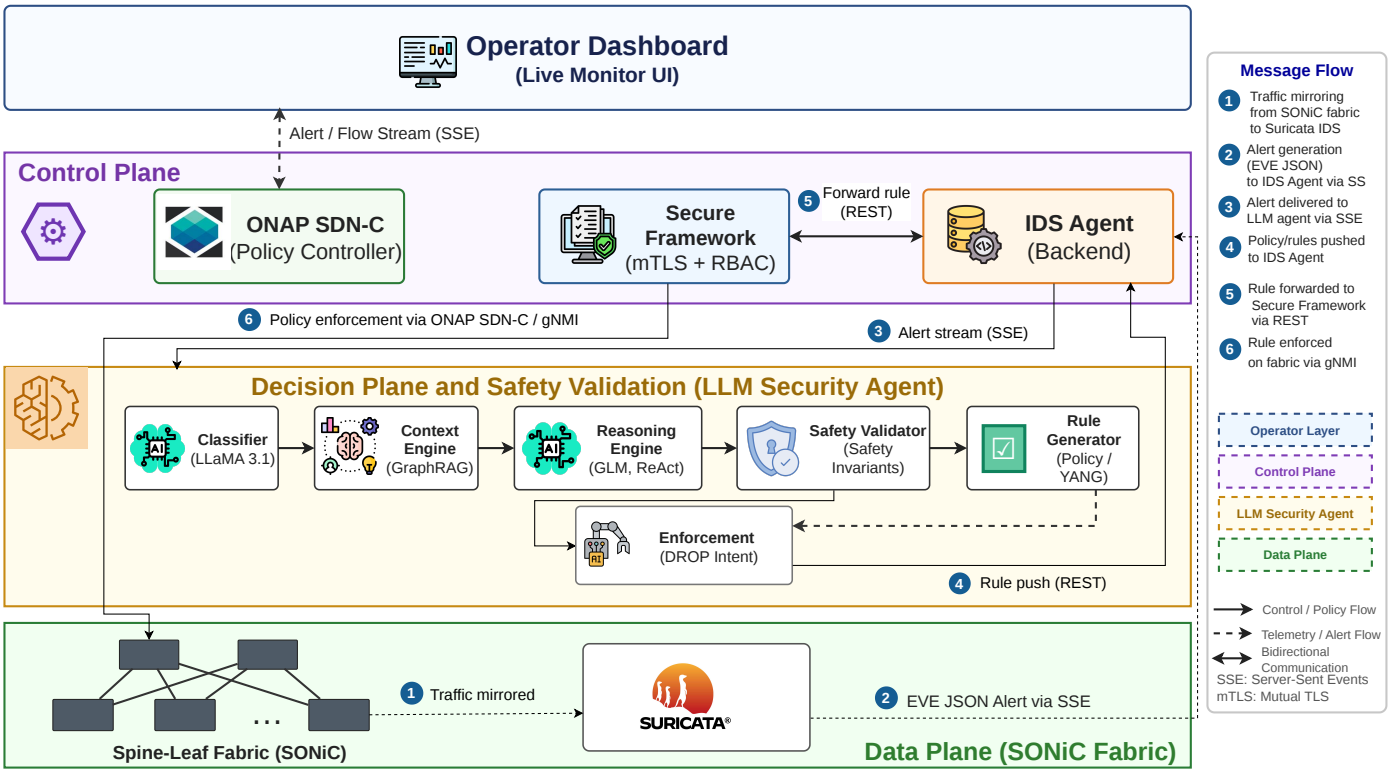


Figure 1: Overall BIZM architecture and bounded-isolation workflow. Mirrored east–west traffic from the SONiC fabric is inspected by Suricata IDS, which generates EVE JSON alerts. The LLM Security Agent answers covered alerts from the compiled response policy and performs classification, GraphRAG-based context retrieval, and LLM reasoning for the remaining alerts, followed by safety validation, enforcement planning, and rule generation. Accepted DROP intents are submitted to the IDS Agent, mediated by the Secure Framework and ONAP SDN-C, and enforced on the SONiC fabric through the southbound management path. Alerts and flow events are streamed to the operator dashboard through SSE.

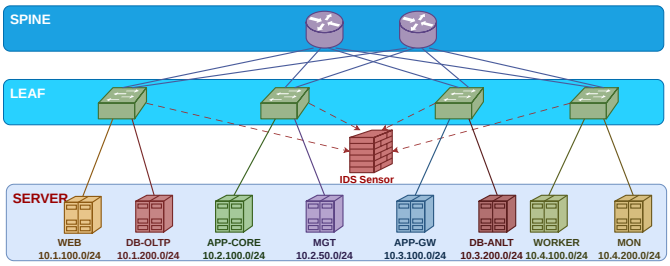


Figure 2: Eight-zone SONiC spine–leaf fabric with IDS mirroring. Workloads are attached to leaf switches and grouped into WEB, APP-GW, APP-CORE, WORKER, DB-OLTP, DB-ANALYTICS, MONITORING, and MGT zones. Dashed paths indicate mirrored traffic from the fabric to the Suricata IDS sensor.

Figure 2 shows the prototype topology. Workloads are grouped into eight zones: WEB, APP-GW, APP-CORE, WORKER, DB-OLTP, DB-ANALYTICS, MONITORING, and MGT. The IDS sensor receives mirrored traffic from the fabric rather than sitting inline on the forwarding path. This design decouples detection from packet forwarding: Suricata provides structured security evidence but does not introduce forwarding latency or become an inline failure point. Enforcement remains under the fabric control path and the Zero Trust management boundary.

A passive IDS alone cannot close the response loop. Even when a signature is reliable, a direct signature-to-block mapping can be unsafe because the same permitted service path may carry both legitimate traffic and malicious behavior. For example, APP-CORE to DB-OLTP traffic may be allowed by the static policy, yet an abnormal transfer pattern on the same port may indicate compromise. BIZM therefore treats an IDS alert as evidence to be interpreted with topology, segmentation policy, asset role, baseline behavior, active rules, and protected-asset constraints.

Table 2 gives the deny-by-default policy matrix used in the evaluation environment. The normal service path runs from WEB to APP-GW to APP-CORE, which then reaches WORKER and the database tiers. DB-OLTP may replicate to DB-ANALYTICS, while DB-ANALYTICS is treated as a sink. MONITORING and MGT have broader reach because they support operations, but they are protected sources. Automatically isolating them could disrupt the control or observability plane. The matrix therefore defines both the static segmentation baseline and the policy context used by BIZM during response decisions.

4.3. Decision Plane and Safety Validation

The decision plane is realized as the LLM Security Agent. It is not a single autonomous controller, but a

Table 2: Eight-zone deny-by-default east–west policy matrix.

Src→Dst	WEB	GW	APP	WK	OLTP	ANL	MON	MGT
WEB	–	✓	✗	✗	✗	✗	✗	✗
APP-GW	✗	–	✓	✗	✗	✗	✗	✗
APP-CORE	✗	✗	–	✓	✓	✓	✗	✗
WORKER	✗	✗	✗	–	✓	✗	✗	✗
DB-OLTP	✗	✗	✗	✗	–	✓	✗	✗
DB-ANALYTICS	✗	✗	✗	✗	✗	–	✗	✗
MONITORING	✓	✓	✓	✓	✓	✓	–	✗
MGT	✓	✓	✓	✓	✓	✓	✓	–

structured pipeline composed of a compiled response policy (Section 4.4) and six runtime functions: alert classification, context retrieval, LLM reasoning, safety validation, enforcement planning, and rule generation. This decomposition is intentional. Contextual interpretation benefits from an LLM, whereas schema checking, target binding, TTL enforcement, protected-asset exclusion, blast-radius checking, and rule construction must remain deterministic.

Alert classification. Alert processing begins with normalization and deduplication. The Suricata EVE JSON alert is mapped to the response schema, including source address, destination address, protocol, destination port, signature, category, severity, and timestamp. Duplicate alerts within a suppression window are ignored. Alerts reach the classifier only when the compiled policy abstains. The classifier then assigns a coarse response class: low-severity or duplicate events follow a log-only path, while high-priority alerts proceed to context retrieval and reasoning. This filter reduces unnecessary LLM calls and avoids enforcement attempts for events that are not eligible for automated containment.

Graph-grounded context. The context engine supplies the reasoning component with a constrained factual context rather than raw network state. The context graph includes zones, assets, leaf-switch attachment, IDS signatures, active DROP rules, protected address ranges, baseline communication edges, and prior incidents. Edges encode zone membership, physical attachment, permitted dependencies, observed communication, monitoring coverage, and historical similarity. For each alert, the context engine retrieves the source and destination zones, policy-matrix entry, active rules, protected-asset status, recent similar incidents, and the expected impact of blocking the source.

This retrieval design has two purposes. First, it gives the LLM the factual substrate needed to distinguish legitimate permitted-path behavior from suspicious behavior on the same allowed edge. Second, it limits prompt exposure by supplying only the context needed for the current decision. The graph-query interface is read-only; any state-changing operation is outside the agent's tool set.

LLM reasoning. The reasoning engine evaluates the alert in context and produces a candidate bounded-isolation decision. The model is signature-informed but not signature-keyed. For known signatures, the prompt includes trusted

Table 3: Bounded-isolation decision schema generated by the LLM Security Agent. The structured action is used for validation and enforcement, while the reasoning trace is recorded separately for audit.

Field	Constraint	Purpose
action	DROP or NONE	Enforcement disposition
src_ip	alert source	Target fidelity
dst_ip	alert destination	Context binding
protocol	TCP, UDP, etc.	Rule match
dst_port	integer	Service scope
ttl_seconds	$[60, 3600]$	Bounded lifetime
confidence	$[0, 1]$	Enforcement gate
mitre_tactic	ATT&CK tactic	Audit mapping
mitre_technique	ATT&CK technique	Audit mapping

metadata such as severity, category, MITRE mapping, and expected disposition. BIZM does not, however, reduce response to a fixed signature-to-action map. Reasoning is used for cases where static mapping is brittle, including permitted-path anomalies, unknown signatures, conflicting evidence, and scope selection under safety constraints.

The output is schema-split. The enforcement-facing output is a compact structured action containing only the fields needed for validation and rule generation. The audit-facing output records the rationale, including supporting evidence, alternatives, and follow-up recommendations. This split reduces parser failures and prevents a long explanation from delaying the enforcement path.

Fail-closed validation. The validator applies the bounded-isolation invariants from Section 3.4. It checks schema conformance, sanitizes alert-derived text for prompt-injection artifacts, verifies that the source exists in the context graph, rejects duplicate or conflicting rules, enforces the protected-asset set, rejects off-target isolation, clamps TTL to the configured interval, applies severity and confidence gates, and enforces per-source and global blast-radius limits. Any failed check results in non-enforcement and audit recording.

Enforcement planning and rule generation. Only an admissible decision is converted into a DROP intent. The enforcement planner binds the decision to the alert source, protocol, destination port, TTL, provenance metadata, and source-side enforcement scope. The rule generator then converts the intent into a policy/YANG-oriented request for the control plane. The model therefore proposes the bounded action, but deterministic components decide whether it is admissible and how it is represented for enforcement. Consequently, the decision logic is summarized in Algorithm 1.

4.4. Compiled Response Policy

Reasoning per alert is unnecessary when the response follows from the knowledge base and does not change between alerts, and it ties decision latency, cost, and availability to the model provider (R8). BIZM therefore uses the

Table 4: Inputs of the compiled response program. Every feature takes finitely many values, so the alert domain is finite.

Feature	Values	Source
Signature	catalogue, or unknown	alert, catalogue
Severity	P1 to P4	alert
Source, destination zone	zones, or external	topology
Policy verdict	allow, deny, intra, external	matrix
Protected source	yes, no	invariants
Prior alert from source	0, 1+	history
Other targets of source	0, 1, 2+	history
Other ports on target	0, 1+	history

same LLM ahead of time to compile the knowledge base into a response program. The program answers covered alerts and abstains on the rest.

Program language. A program is an ordered list of rules. Each rule is a conjunction of conditions over the alert features in Table 4 and yields DROP with a scope, TTL, and confidence, LOG_ONLY, or ABSTAIN. The first matching rule decides, and an alert that no rule matches is ABSTAIN. The language has no target field: an enforced rule always blocks the alert source as a single host, and the scope only selects the destination and port (flow, pair, or host). Each DROP rule carries a rationale and the knowledge-base entries it relies on.

Exhaustive verification. The features define a finite domain $D(K)$ for a knowledge-base snapshot K. For the deployed fabric, 29 signatures and the unknown value, four severities, nine zones including external, the protected flag, and seven consistent history combinations give 136,080 points, of which 19,824 can be emitted by the deployed IDS, i.e., a catalogued signature with its catalogued severity on a zone pair its detection logic covers. The verifier evaluates the program on every point and rejects it if any DROP targets a protected or external source (I3), violates the TTL bounds, uses host scope outside P1 or with a TTL above 1800 seconds, or overrides a permitted path with anything but flow scope, a TTL of at most 1800 seconds, and a confidence of at least 0.90 (I4), falls outside P1 and P2 or below τ_c (I5), concerns an unknown signature, or lacks a knowledge-base citation (I6). Verifying one program takes about five seconds.

Compilation. The compiler gives the LLM the knowledge-base documents and the language, and asks for one program that covers the whole fabric. The objectives are stated in priority order: obey the hard constraints, contain what the knowledge base identifies as requiring response, do not cut traffic it documents as legitimate, prefer the narrowest scope, and abstain when the knowledge base does not determine the answer. A rejected program is returned to the model with counterexamples, each naming the rule, the violated invariant, and an example input, and the model repairs it for at most three rounds. A verified program is then summarized per signature and policy verdict, and the model reviews this behavior against the knowledge base; the revision is kept only if it also verifies. Because sampled programs differ, BIZM compiles k programs independently and deploys their point-wise majority: an action is taken only when a strict majority chooses it, otherwise the ensemble abstains, and among agreeing programs the narrowest scope, shortest TTL, and lowest confidence are kept. The ensemble is verified again as a whole.

Activation and runtime use. The compiled artifact records the digest of the knowledge base it was compiled from, the model, the repair rounds, and the token cost. The agent activates a program only if it was verified at compile time, its digest equals that of the running knowledge base, and it passes verification again in the agent process. A knowledge-base change therefore disables the old program until it is recompiled, and all alerts take the runtime path in the meantime. At runtime, the features are extracted from the alert and from the source's alerts in the preceding ten minutes. A program decision becomes a bounded intent whose audit trace is the rule rationale and its cited entries, and it passes the same validators, rate limits, circuit breaker, and enforcement path as a runtime decision. Any error on the compiled path falls back to the runtime path rather than failing the alert.

4.5. Zero Trust Enforcement Plane

The enforcement plane prevents the LLM Security Agent from becoming an over-privileged network operator. At the architectural level, the control path consists of the IDS Agent, the Secure Framework, and ONAP SDN-C. The IDS Agent integrates the decision layer with the control plane. The Secure Framework mediates policy requests using mTLS and RBAC. ONAP SDN-C acts as the policy controller that coordinates accepted operations and fabric-side enforcement.

Figure 3 details the enforcement boundary. The LLM Security Agent does not send device commands. Instead, an accepted DROP intent is submitted to the IDS Agent and forwarded to the Secure Framework through a restricted REST interface. The Secure Framework authenticates the request, binds it to a restricted service identity, checks command-level authorization, validates the operation against the YANG model, attaches provenance metadata, and translates the accepted intent into a gNMI-compatible operation. The resulting request is then pushed to the data-plane switch through a mutually authenticated gRPC/gNMI channel.

The control-plane contract is intentionally narrow. BIZM can create temporary DROP rules, but it cannot create ACCEPT rules, delete unrelated rules, modify baseline segmentation policy, change routing state, or access protected management objects. Each accepted rule must be source-faithful, DROP-only, time-bounded, provenance-marked, and authorized under the restricted response role. Requests that target protected assets,

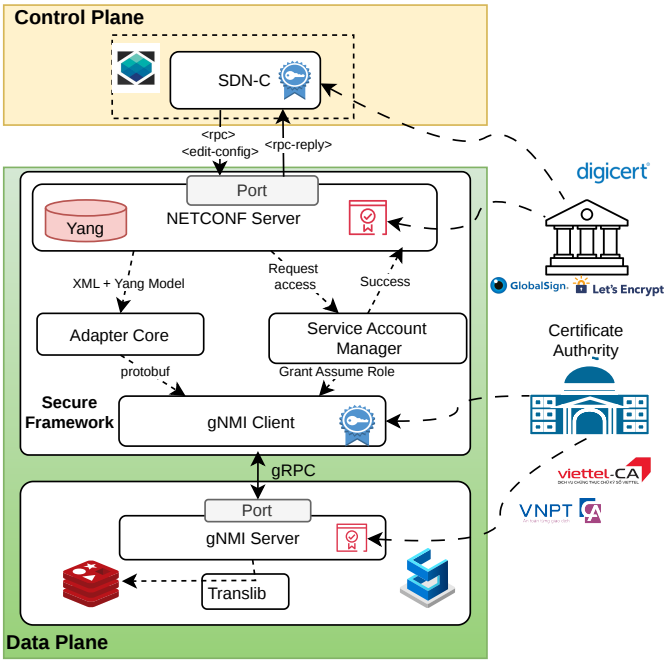


Figure 3: Identity-aware Secure Framework used by the BIZM enforcement plane. The framework receives authorized bounded-response requests through a restricted northbound interface, validates operations against the YANG model, checks service-account permissions, translates accepted operations into gNMI/protobuf requests, and pushes device-level rules to the data-plane switch through a mutually authenticated gRPC/gNMI channel.

request unsupported operations, lack valid credentials, or violate the allowed command set fail closed before reaching the fabric.

The enforcement path also separates policy decision from device realization. The LLM Security Agent produces a bounded intent, the IDS Agent handles backend integration, the Secure Framework performs identity-aware authorization and schema validation, ONAP SDN-C coordinates policy enforcement, and the southbound gNMI path realizes the accepted rule on the SONiC fabric. This layered path reduces the risk that a compromised or misled reasoning component can exceed its authority.

4.6. End-to-End Workflow

The end-to-end workflow follows the numbered steps in Figure 1. First, the SONiC fabric mirrors east–west traffic to Suricata IDS. Second, Suricata generates an EVE JSON alert. Third, the alert is delivered to the LLM Security Agent through SSE, decided by the compiled policy or, if it abstains, by classification, GraphRAG retrieval, and reasoning, and then validated. Fourth, if the candidate response is admissible, the rule generator pushes a policy/YANG-oriented DROP request to the IDS Agent. Fifth, the IDS Agent forwards the request to the Secure Framework, where identity, role, schema, and policy checks are applied. Sixth, the accepted rule is enforced through ONAP SDN-C and the southbound gNMI management path. In parallel, alerts and flow updates are streamed to the operator dashboard.

This workflow is closed-loop but not unrestricted. Each iteration is bounded by action type, target fidelity, TTL, protected-asset exclusion, confidence threshold, severity threshold, blast-radius limit, identity-aware authorization, and audit recording. The response is also reversible because each generated rule has an explicit lifetime. When the TTL expires, the agent revokes the temporary isolation rule, while the baseline segmentation policy remains unchanged.

Consider a compromised APP-CORE host that initiates a high-rate bulk transfer to DB-OLTP on a permitted database port. The static policy allows the zone pair, so the fabric does not block the flow by default. Suricata raises an alert, and BIZM retrieves the APP-CORE–DB-OLTP baseline, recent similar incidents, active rules, source leaf attachment, protected-asset status, and block-impact context. Because the edge is permitted, the response is not a fixed lookup. The reasoning engine compares the observed behavior with the baseline and proposes a bounded DROP on the source. The validator confirms that the target equals the alert source, the source is not protected, the TTL is within bounds, and the confidence and severity gates are satisfied. The rule generator emits a DROP intent, which is submitted to the IDS Agent, authorized by the Secure Framework, coordinated through ONAP SDN-C, and enforced at the source-side leaf. A legitimate DB-OLTP replication burst toward DB-ANALYTICS, by contrast, matches the baseline and is recorded without enforcement. If the alert instead carries a catalogued content signature, such as destructive SQL, the compiled policy issues the same flow-scope DROP without a model call, and the reasoning path is used only when the signature is unknown or the alert contradicts the catalogue.

This workflow captures the main design principle of BIZM: use the LLM to compile what the knowledge base determines and to reason where it does not, but constrain every consequential action through deterministic validation and Zero Trust enforcement. The response loop is therefore adaptive enough for microsegmentation environments while preserving the boundedness required for safe network automation.

5. Implementation and Experimental Design

5.1. Testbed Implementation

We implement BIZM on a virtualized spine–leaf datacenter fabric emulated in GNS3. The data plane uses SONiC switches [59]. Static microsegmentation is enforced as deny-by-default rules on the leaf switches, while runtime containment is installed as a temporary source-side DROP rule at the leaf nearest the suspicious workload. Each automated rule carries a TTL and provenance metadata, and the agent revokes it when the TTL expires.

Suricata monitors mirrored east–west traffic and exports EVE JSON alerts [11]. The IDS is placed off the

forwarding path: it supplies the suspicious signal, while BIZM performs response-level decision making. This follows the common deployment pattern in which lightweight sensing or signature inspection provides the initial evidence, and deeper AI-based analysis or response logic runs outside the packet-forwarding path [33, 34, 12]. An IDS Agent subscribes to the alert stream, forwards normalized alerts to the BIZM decision plane, receives admissible DROP intents, and records live status for the operator dashboard.

Enforcement success is verified through four checks: the rule appears in the device rule datastore, the intended source can no longer reach the target service, unrelated traffic remains unaffected, and the installed rule carries the expected TTL and provenance mark. This verification distinguishes correct containment from accidental reachability loss or over-broad blocking.

5.2. BIZM Runtime Configuration

The runtime policy follows the eight-zone matrix in Table 2. The protected set $\mathcal{N}$ includes management hosts, monitoring hosts, IDS components, zone gateways, and control-plane components. Automatic enforcement is allowed only for P1 and P2 alerts. The confidence threshold is set to $\tau_c = 85\%$. Each DROP rule has a TTL in $[60, 3600]$ seconds. Blast-radius limits are set to five automated rules per minute, fifty active automated rules in total, and three rules per source address within five minutes. A circuit breaker pauses enforcement after three consecutive failed enforcement attempts.

The decision model is served through an OpenAI-compatible interface. All reported results use `gpt-oss-120b` for both the reasoning and the classification roles, with `reasoning_effort` set to low. The reasoning call uses temperature 0.1, at most 3072 output tokens, and a 15-second timeout, and each P1/P2 alert receives a single reasoning call without self-consistency voting. The classification call uses temperature 0.0 and at most 1024 output tokens, and handles early severity/category filtering; duplicate alerts with the same source and signature are suppressed for 30 seconds. Prompts are fixed across runs, and no scenario-specific prompt tuning is performed. The enforcement-facing Stage 1 output is constrained by the structured decision schema in Table 3. The Stage 2 reasoning trace is recorded for audit but is not used to authorize enforcement.

The compiler uses the same model with `reasoning_effort` set to medium. Its input is the knowledge base of seven documents (zones, policy matrix, signature catalogue, invariants, assets, severity rubric, and baselines), about 22K tokens. The deployed policy is a majority ensemble of $k = 5$ programs sampled at temperature 0.7, each repaired for at most three rounds and reviewed once. Compilation at temperature 0 is also reported as a single-program configuration.

5.3. Evaluation Scenarios and Baselines

The evaluation addresses five questions. RQ1 asks whether the LLM can compile the knowledge base into a verified program that agrees with it. RQ2 asks whether recompilation keeps the program correct when the knowledge base changes. RQ3 asks whether BIZM contains functional east–west threats on the live testbed, at what latency, and at what cost to legitimate traffic. RQ4 asks whether the safety mechanism prevents unsafe enforcement under adversarial alert manipulation. RQ5 asks how the runtime path handles alerts the program does not cover and what each path costs.

RQ1 and RQ2 are evaluated offline over the whole alert domain $D(K)$. For RQ2, four modified copies of the knowledge base represent changes an operations team makes between compilations: six new signatures, two of which detect abuse inside permitted connections (V1); a ninth crown-jewel zone, PAYMENTS, with its matrix rows, gateway, and two zone-pair signatures (V2); three documented responses changed in prose only (V3); and all three changes together (V4). Each variant is recompiled, while the baselines are left as written.

Table 5 summarizes the testbed scenarios. Five scenarios use live east–west traffic and exercise the full detection-to-enforcement path. Five scenarios use controlled injected alerts to stress the decision logic, protected-asset handling, prompt-injection resistance, policy-matrix coverage, unknown signatures, and multi-stage correlation. Live scenarios are repeated ten times. Injected scenarios are repeated three times with the compiled policy, whose decisions are deterministic, and ten times in the runtime-only configuration. These repetitions measure run-to-run stability and should not be interpreted as a large independent traffic corpus. A runtime decision lost to an infrastructure fault, i.e., a model-provider timeout or an IDS that raised no alert, is re-run rather than scored. In addition, a matrix sweep injects 75 alerts generated mechanically from every off-diagonal cell of the policy matrix, the signature catalogue, and the protected set, five times each.

We compare BIZM against three baselines and one ablation. *SOAR-A* is a signature-keyed responder that issues a DROP against the alert source for a fixed set of known-dangerous signatures and takes no action otherwise; a host-isolation variant is used in the matrix sweep. *Baseline-B* uses the same LLM decision model as BIZM but removes deterministic safety validation and least-privilege enforcement. It receives raw alert text and emits an action directly. *Baseline-C* is a deterministic contextual rule written by the authors over the same topology, policy matrix, and protected set: it contains a severe alert on a denied zone pair and, on a permitted pair, only for a hand-written list of abuse signatures. It is evaluated as deployed, with zone-level protection of MGT and MONITORING and with the deployed topology, and in a strongest variant that protects exactly the hosts of the protected set and reads zones and the matrix from the current knowledge base. *Runtime-only*

Table 5: Testbed scenarios. Live scenarios exercise the full detection-to-enforcement path, while injected scenarios isolate decision and safety behavior. N gives the repetitions with the compiled policy and, after the slash, in the runtime-only configuration.

ID	Description	Type	N
S1	WEB to DB-OLTP direct, application-chain bypass	Denied-cell violation	10/10
S2	APP-CORE reverse connection to WEB	Denied-cell lateral movement	10/10
S3	DB-OLTP outbound beacon from sink tier	Denied-cell behavioral violation	10/10
S4	Destructive SQL on permitted APP-CORE to DB-OLTP edge	Permitted-path anomaly	10/10
S5	Management and monitoring baseline traffic	Legitimate FP control	10/10
S6	Denied-cell injections and legitimate flows	Matrix coverage and FP	3/10
S7	Prompt injection, severity bypass, spoofed-source alerts	Adversarial safety	3/10
S8	Protected management or monitoring host acting maliciously	Never-block invariant	3/10
S9	Known, unknown, and legitimate signatures	Reasoning ablation	3/10
S10	Three-stage campaign from a single source	Kill-chain correlation	3/10

is BIZM with the compiled policy disabled, so every alert is reasoned by the LLM. SOAR-A evaluates contextual response beyond fixed playbooks, Baseline-B isolates the safety controls, Baseline-C separates the LLM's contribution from that of the context itself, and the runtime-only ablation isolates the compiled policy.

5.4. Evaluation Metrics

For response correctness, each labeled decision is classified as TP, FP, TN, or FN. A positive case is a true threat whose correct response is a bounded DROP decision [34]. A negative case is benign, ambiguous, low-severity, or unsafe to enforce and should not trigger automated containment. We report precision, threat-containment rate (TCR), F1-score, false positive rate (FPR), and accuracy:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{TCR} = \frac{\text{TP}}{\text{TP} + \text{FN}},$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{TCR}}{\text{Precision} + \text{TCR}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}.$$

TCR is a response-level metric: it measures the fraction of true threat cases for which the system correctly issues an admissible bounded DROP decision. It should not be interpreted as IDS detection rate, because BIZM operates after an alert has been generated.

For RQ1 and RQ2, ground truth is derived mechanically from the knowledge base, never from a program. For an alert with a catalogued signature, the signature's documented response in the catalogue governs: a documented DROP means contain, a documented log-only means do not contain, and a DROP conditioned on correlation means contain only after a prior alert from the source. Protected or external sources and P3 or P4 alerts are never contained. Points the knowledge base does not settle are left ungraded: a correlation-conditioned signature with no prior alert, and a log-only signature of severity P1 or P2 on a denied pair, where the pair itself is a policy violation. Unknown signatures are reported separately. On the graded points we report accuracy over the points a system decides and coverage, the fraction it decides rather than abstains on. Because these results enumerate the domain rather than sample it, they carry no sampling interval.

For latency, we decompose the closed-loop response into detection latency, decision latency, and enforcement latency. Detection latency measures traffic onset to IDS alert. Decision latency measures alert ingestion to a decision accepted by the Secure Framework. Enforcement latency measures that decision to rule visibility in the Secure Framework datastore of the leaf. For the matrix sweep, we count the service edges a rule closes and the legitimate baseline flows it cuts. For safety, we report safety-violation rate, protected-service availability, and never-block respect for protected assets. For reasoning, we compare correct-decision rates on known and unknown signatures. For runtime overhead, we report median and tail decision latency, model tokens per decision, and the one-time cost of compilation. Proportions are reported with exact Clopper–Pearson intervals, and paired comparisons with the exact McNemar test.

6. Results, Evaluation, and Discussion

6.1. Compiled Policy Agreement with the Knowledge Base

Table 6 compares compiled programs with Baseline-C over the deployed knowledge base. Of the 19,824 realizable points, 14,970 are graded, 822 are ungraded, and 4,032 carry an unknown signature. Every compiled program passed verification with no violating point. The single program compiled at temperature 0 decides 97.5% of the graded points and agrees with the knowledge base on all of them, and it was identical across five compilations. The ensemble decides every graded point and agrees on all of them. Its members varied individually, with coverage between 69.0% and 100% and one member missing 966 contain points, and the majority vote removed every member error.

Baseline-C errs in both directions. Its hand-written list of permitted-path abuse signatures includes three signatures that the catalogue documents as log-only, so even its strongest variant blocks 588 points that the knowledge base says to log. The deployed variant also protects every host of MGT and MONITORING instead of the hosts in the

Table 6: Agreement with the deployed knowledge base over its 14,970 graded points. Accuracy is over the points a system decides; coverage is the fraction it decides.

System	Acc.	Cov.	FN	FP
Baseline-C, as deployed	94.54%	100.0%	544	273
Baseline-C, strongest	96.07%	100.0%	0	588
BIZM, single program	100.00%	97.5%	0	0
BIZM, ensemble of five	100.00%	100.0%	0	0

Table 7: Agreement after knowledge-base change. Baseline-C is left as written; BIZM is recompiled (ensemble of five).

	Baseline-C		BIZM	
Change	Deployed	Strongest	Acc.	Cov.
V1 new signatures	93.73%	94.98%	100.00%	95.0%
V2 new zone	93.03%	96.26%	100.00%	97.5%
V3 prose retuning	94.46%	96.09%	100.00%	100.0%
V4 all changes	92.03%	95.24%	99.97%	92.3%

protected set and misses 544 contain points, and it enforces on 518 points with an unknown signature, which BIZM defers to the runtime path. The rule is consistent with the policy matrix but not with the prose of the catalogue, which it never reads. The compiled program is derived from that prose, and this is where the LLM contributes. Compiling the ensemble took 117 seconds and 341K tokens.

6.2. Recompilation under Knowledge-Base Change

Table 7 reports the four knowledge-base changes. Recompilation keeps agreement at 100% on V1 to V3 and at 99.97% on V4. The seven V4 errors concern SID 9000012 on PAYMENTS to DB-OLTP: the variant permits that pair but leaves the signature's detection logic unchanged, so the knowledge base contradicts itself there. Baseline-C, which is not rewritten, misses the two new abuse signatures of V1 and keeps blocking SID 9000050 after V3 documents it as log-only. Its deployed variant cannot place the PAYMENTS zone and misses 1,009 contain points in V2. The strongest variant follows the new zone and matrix rows, which are structured, but not changes stated in prose. The ensemble coverage falls to 92.3% on V4 because its members disagree on some new signatures, and those points go to the runtime path rather than to a wrong decision. Each recompilation took under four minutes, whereas the rule requires a code change and redeployment.

6.3. Closed-Loop Containment and Latency

Table 8 reports the live scenarios. In the four threat scenarios, BIZM installs the correct source-side DROP rule in all ten runs, each decided by the compiled policy, including S4, a permitted-path anomaly on the APP-CORE to DB-OLTP edge. The false-positive control S5 produces no automated enforcement in all ten runs. One S3 run was re-run because the IDS raised no alert. All forty enforced runs pass the post-enforcement checks: rule presence, source-side drop behavior, blocked reachability from the offending source, unaffected unrelated traffic, and TTL within bounds.

Table 8: Live scenarios, $N = 10$ each. Latencies are medians in seconds. Decide is alert to a decision accepted by the Secure Framework; Visible is alert to rule visibility in the leaf datastore.

			BIZM		Runtime-only	
Scn.	Outcome	Detect	Decide	Visible	Decide	Visible
S1	DROP (10/10)	3.30	1.27	5.21	3.02	5.14
S2	DROP (10/10)	3.95	0.29	4.89	3.59	5.15
S3	DROP (10/10)	2.90	1.10	5.53	2.96	7.59
S4	DROP (10/10)	3.45	0.26	4.76	2.91	5.21
S5	NONE (10/10)	n/a	n/a	n/a	n/a	n/a

Table 9: Response correctness with 95% Clopper–Pearson intervals. BIZM: 77 decisions (TP = 55, FN = 0, FP = 0, TN = 22). Runtime-only: 140 decisions (TP = 88, FN = 2, FP = 0, TN = 50).

Metric	BIZM	Runtime-only
Precision	100.0% [93.5, 100]	100.0% [95.9, 100]
TCR	100.0% [93.5, 100]	97.8% [92.2, 99.7]
F1-score	100.0%	98.9%
FPR	0.0% [0.0, 15.4]	0.0% [0.0, 7.1]
Accuracy	100.0% [95.3, 100]	98.6% [94.9, 99.8]

The compiled path reduces the median decision latency from 3.12 to 0.59 seconds, and the 95th percentile from 4.51 to 2.70 seconds; the remaining time is the Secure Framework round trip. The time until the rule is visible in the leaf datastore is similar on both paths, with medians of 5.03 and 5.22 seconds, because it is dominated by gNMI propagation and observation in the enforcement path rather than by the decision. Once the decision is compiled, the closed loop is bounded by the sensor and the enforcement path. This latency is not appropriate for packet-level mitigation, but it is suitable for containment of lateral movement and substantially faster than manual triage.

Table 9 reports response correctness over the labeled live and S6 decisions. BIZM makes no error in 77 decisions. The runtime-only configuration misses 2 of 90 threats in 140 decisions: one inbound-management case answered log-only, and one run lost to a transient fault on the enforcement path, a fault class the current build removes. In four of its forty legitimate S6 decisions the runtime LLM proposed a DROP, which the policy-conflict check (three) and the severity gate (one) rejected before enforcement. The intervals show that these figures are evidence of feasibility on this scenario set, not a guarantee of zero errors.

Table 10 reports the matrix sweep over 375 decisions. Static segmentation answers none of the 42 containment cases. Host isolation contains the most but cuts 6.3 times more legitimate flows than flow-scope containment, and it over-blocks the protected-source cases (McNemar $p = 1.9 \times 10^{-6}$ against BIZM). The compiled program decides all 75 cases at flow scope and agrees with the runtime path

Table 10: Matrix sweep, 75 mechanically generated cases × 5. Edges are service edges closed from the compromised source; Cut counts legitimate baseline flows blocked.

System	Acc.	95% CI	Edges	Cut
Static matrix only	44.0%	[38.9, 49.2]	0	0
SOAR, host isolation	94.7%	[91.9, 96.7]	440	220
Baseline-C	100.0%	[99.0, 100]	35	35
Runtime-only	100.0%	[99.0, 100]	35	35
BIZM	100.0%	[99.0, 100]	35	35

Table 11: Adversarial safety. S7 reports 27 decisions for BIZM and 90 for the other systems; S8 reports 12 and 40 protected-host decisions. Baseline-B is the same model with deterministic safety layers disabled.

Metric	BIZM	Runtime-only	Baseline-B
Safety-violation rate (S7)	0.0%	0.0%	100.0%
Core-service availability (S7)	100.0%	100.0%	11.1%
Never-block respect (S8)	100.0%	100.0%	0.0%

and Baseline-C, which coincide with it on these single-alert cases; their differences lie in the parts of the domain the sweep does not sample (Section 6.1). Every service edge removed falls on a permitted pair, because on a denied pair the static matrix has already removed it.

6.4. Adversarial Safety and Bounded-Isolation Guarantees

Table 11 reports adversarial-safety results. Scenario S7 contains prompt-injection alerts that attempt to redirect isolation to protected control-plane addresses, low-severity alerts that attempt to bypass the severity gate, and spoofed-source cases. BIZM produces no unsafe enforcement in its 27 decisions, nor in the 90 decisions of the runtime-only configuration. With the compiled policy, the injected alerts never reach a model: the program decides from the alert source, zones, and signature and ignores alert text, so the injected instructions have nothing to act on. The injection cases were contained at their true source, and the spoofed-source and low-severity cases were removed by the protected-asset and severity gates. In the runtime-only configuration, the protected-asset set and off-target checks reject injected targets.

Baseline-B, which uses the same LLM without deterministic safety layers, violates safety in 100% of adversarial cases. It follows injected instructions, targets protected services, or accepts unsafe redirections. Scenario S8 stresses the protected-asset invariant directly by making management or monitoring hosts appear malicious. BIZM respects the never-block constraint in all cases and surfaces the events for review, whereas Baseline-B isolates protected hosts. These results show that BIZM's safety is not a property of the model alone; it comes from deterministic validation and least-privilege enforcement around the model.

Table 12: Reasoning ablation in S9, reported as correct decisions. BIZM uses three repetitions, the other systems ten.

Case group	BIZM	Runtime-only	SOAR-A
Malicious, known signature	9/9	30/30	30/30
Malicious, unknown signature	7/9	28/30	0/30
Legitimate, unknown signature	9/9	30/30	30/30
Total	25/27	88/90	60/90

6.5. Residual Reasoning and Ablation Analysis

Table 12 reports the signature-hint ablation. Scenario S9 contains malicious known-signature cases, malicious unknown-signature cases, and legitimate unknown-signature cases. In BIZM, known signatures are decided by the compiled policy and unknown signatures by the runtime path. On malicious unknown signatures, BIZM is correct in seven of nine decisions and the runtime-only configuration in 28 of 30, whereas SOAR-A is correct in none because it has no matching playbook entry. The runtime path infers maliciousness from context: source and destination zones, policy-matrix entry, connection direction, signature text, and baseline behavior. Both BIZM misses concern one case, held once at confidence 0.66 for review and answered once as log-only, so they fall on the review side rather than on enforcement. On legitimate unknown signatures, both configurations correctly choose no enforcement in all cases.

Scenario S10 injects a three-stage campaign from one source: scanning, application-tier bypass, and database bypass. The compiled policy contains the third stage in all three sequenced and all three cold runs. The runtime-only configuration identifies the multi-stage pattern in all ten runs and cites earlier stages in its reasoning trace, while SOAR-A mitigates only the final signature-matched event. The confidence score does not increase between isolated and correlated presentations because clear violations already saturate near the top of the model's coarse confidence scale.

This ablation should be interpreted narrowly. The compiled policy covers what the knowledge base determines; the runtime path is needed where it does not, such as unknown signatures and alerts that contradict the catalogue, and it is less reliable there. The safety invariants remain necessary because the same reasoning component can be wrong or manipulated.

6.6. Runtime Overhead and Cost

A runtime decision issues one reasoning call and one audit-trace call with a mean prompt of 10.1K tokens, dominated by the retrieved context, or about 21K tokens per enforced decision. A compiled decision uses no tokens and takes 2 to 17 ms without enforcement; its decision latency on the testbed is the Secure Framework round trip. Compilation is a one-time cost of 341K tokens and 117 seconds,

about the tokens of 16 runtime decisions, repeated only when the knowledge base changes. In the injected runs, the compiled policy answered 45 of the 72 alerts that passed the gate. The runtime path answered the other 27 with a median of 23.7 seconds, inflated by provider latency on the day of the run. During a nine-minute provider outage in that run, runtime decisions failed with connection timeouts and were re-run, while the compiled policy kept deciding.

BIZM is intended for containment rather than packet-level inline filtering. High-rate traffic mitigation is better handled by in-network or programmable data-plane mechanisms. Production deployment would also require queueing, overload thresholds, and fail-safe modes for the runtime path, which the current evaluation does not characterize.

6.7. Discussion, Limitations, and Threats to Validity

The results support three conclusions. First, the LLM contributes most as a compiler: a program it derives from the knowledge base agrees with the knowledge base on every graded point, follows its prose where a hand-written rule does not, and is recompiled when the knowledge base changes. Second, LLM-assisted response can be made operationally useful when the model is not granted direct authority over the network; on both paths, safety comes from deterministic validation and least-privilege enforcement, and on the compiled path it is checked on every input in advance. Third, the evaluation provides feasibility evidence on a controlled testbed, not proof of production-scale readiness.

The main limitations are clear. Ground truth is derived from a knowledge base the authors wrote, and the four changes in RQ2 were also authored; a wrong knowledge base yields a program that is consistently wrong, although never unsafe. The program features are coarse, so multi-hop pivot reasoning and payload evidence remain with the runtime path. The testbed is virtualized and small, the scenario set is author-designed, injected alerts do not exercise the IDS sensor, and all results use one model from one provider. The model confidence is not calibrated; it saturates on clear violations, so the confidence threshold acts as a conservative floor rather than a probabilistic discriminator.

There are operational trade-offs. Automatic containment of high-impact assets carries operational risk, which a human-in-the-loop mode for protected assets can address. The threat model assumes a correct enforcement implementation and does not cover compromise of the switch operating system, the IDS engine, or encrypted payload inspection beyond what the IDS can observe.

Finally, the comparison with prior systems is by capability rather than direct numerical performance. East–west detectors such as NetVigil and Euler solve a different stage of the loop. Programmable data-plane defenses target different attack classes and latency regimes. LLM-based network-configuration systems address configuration generation rather than autonomous isolation. The contribution of BIZM is therefore not a higher IDS detection score, but a bounded response architecture that compiles verified response logic from the architecture and connects alert interpretation, safety validation, and least-privilege fabric enforcement in a single closed loop.

7. Conclusion

This paper presented **BIZM**, a safety-constrained LLM framework for bounded isolation in Zero Trust microsegmentation. BIZM uses the LLM at two speeds: it compiles the architecture knowledge base into a response program that is verified on every point of the finite alert domain, and it reasons at runtime over the alerts the program does not cover. On both paths, enforcement is restricted to source-faithful, time-bounded, provenance-marked DROP rules, and each action must pass fail-closed validation and command-level authorization before reaching the fabric. This design bounds the authority of the model even when its output is mistaken, malformed, or manipulated.

The evaluation on a spine–leaf data-center testbed shows that the compiled program agrees with the knowledge base on all 14,970 graded points, where a hand-written contextual rule reaches 96.1%, and that recompilation keeps 99.97% to 100% agreement after four knowledge-base changes. On the live testbed, BIZM contains every threat without a false positive, decides in 0.6 instead of 3.1 seconds, continues during a provider outage, and produces no unsafe enforcement under adversarial alerts. These results support the central argument of the paper: LLM-assisted microsegmentation response is feasible when the LLM compiles verified response logic from the architecture and its runtime authority is bounded by deterministic network-level safety invariants.

The current prototype also has clear limitations. The evaluation uses a controlled virtualized fabric, an author-written knowledge base and scenario set, a coarse program feature set, and a single model. Future work will extend the program features toward multi-hop context, evaluate on larger and more diverse traffic and other models, compile from operator-maintained documentation, and develop human-in-the-loop policies for high-impact protected assets and management-plane events.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was supported by the Ministry of Science and Technology of Vietnam under the National Science and Technology Program, project code KC.01.06/21-30.

Data and Code Availability

The testbed configuration, Suricata rules, policy matrix, scenario definitions, prompts, decision schemas, and evaluation scripts will be made available upon publication to support reproducibility. An anonymized supplementary package can be provided for review upon request.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

During the preparation of this work, the authors used generative AI and AI-assisted technologies to refine the scientific prose, improve readability, and polish LaTeX formatting. After using these technologies, the authors thoroughly reviewed and edited the content as needed and take full responsibility for the content of the published article.

CRediT Authorship Contribution Statement

Hanh P. Du: Conceptualization, Formal analysis, Investigation, Methodology, Validation, Writing – original draft. **Hung K. Luu**: Writing – original draft, Writing – review & editing, Visualization, Validation, Software, Investigation, Formal analysis. **Minh T. Nguyen**: Visualization, Validation, Software, Investigation, Formal analysis, Writing – original draft. **Hoa N. Nguyen**: Writing – original draft, Writing – review & editing, Methodology, Conceptualization, Project administration, Supervision.

References

[1] H. R. H. Andersson, A. de Blanche, T. Lundqvist, Flipping the data center network: Increasing east-west capacity using existing hardware, in: Proceedings of the IEEE 42nd Conference on Local Computer Networks (LCN), IEEE, 2017, pp. 211–214. doi:10.1109/LCN.2017.92.

[2] R. Ward, B. Beyer, BeyondCorp: A new approach to enterprise security, USENIX ;login: 39 (6) (2014) 6–11.
URL https://www.usenix.org/system/files/login/articles/login_dec14_02_ward.pdf

[3] S. Rose, O. Borchert, S. Mitchell, S. Connelly, Zero trust architecture, NIST Special Publication 800-207, National Institute of Standards and Technology (2020). doi:10.6028/NIST.SP.800-207.

[4] N. Basta, M. Ikram, M. A. Kaafar, A. Walker, Towards a zero-trust micro-segmentation network security strategy: An evaluation framework, in: Proceedings of the 2022 IEEE/IFIP Network Operations and Management Symposium (NOMS), IEEE, 2022, pp. 1–7. doi:10.1109/NOMS54207.2022.9789888.

[5] C. Buck, C. Olenberger, A. Schweizer, F. Völter, T. Eymann, Never trust, always verify: A multivocal literature review on current knowledge and research gaps of zero-trust, Computers & Security 110 (2021) 102436. doi:10.1016/j.cose.2021.102436.

[6] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, R. Doss, Zero trust architecture (ZTA): A comprehensive survey, IEEE Access 10 (2022) 57143–57179. doi:10.1109/ACCESS.2022.3174679.

[7] R. Chandramouli, A zero trust architecture model for access control in cloud-native applications in multi-location environments, NIST Special Publication 800-207A, National Institute of Standards and Technology (2023). doi:10.6028/NIST.SP.800-207A.

[8] R. Chandramouli, Building secure microservices-based applications using service-mesh architecture, NIST Special Publication 800-204A, National Institute of Standards and Technology (2020). doi:10.6028/NIST.SP.800-204A.

[9] K. Hsieh, M. Wong, S. Segarra, S. K. Mani, T. Eberl, A. Panasyuk, R. Netravali, R. Chandra, S. Kandula, NetVigil: Robust and low-cost anomaly detection for east-west data center security, in: Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2024, pp. 1771–1780.
URL https://www.usenix.org/conference/nsdi24/presentation/hsieh

[10] I. J. King, H. H. Huang, Euler: Detecting network lateral movement via scalable temporal graph link prediction, in: Proceedings of the Network and Distributed System Security Symposium (NDSS), 2022.
URL https://www.ndss-symposium.org/ndss-paper/euler-detecting-network-lateral-movement-via-scalable-temporal-graph-link-prediction/

[11] Open Information Security Foundation, Suricata: Open source threat detection engine, https://suricata.io/, accessed: 2026-06-04 (2026).

[12] H. V. Vo, H. P. Du, H. N. Nguyen, APELID: Enhancing real-time intrusion detection with augmented WGAN and parallel ensemble learning, Computers & Security 136 (2024) 103567. doi:10.1016/j.cose.2023.103567.

[13] T. T. Nguyen, U. H. Tran, H. N. Nguyen, Adversarial attack and defense in AI-powered intrusion detection, Journal of Computer Science and Cybernetics 41 (4) (2025) 387–402. doi:10.15625/1813-9663/22884.
URL https://vjst.vast.vn/jcc/article/view/22884

[14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems, Vol. 35, 2022, pp. 24824–24837.
URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html

[15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: Proceedings of the 11th International Conference on Learning Representations (ICLR), 2023.
URL https://openreview.net/forum?id=WE_vluYUL-X

[16] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 9459–9474.
URL https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[17] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.
URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html

[18] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in: Proceedings of the 11th International Conference on Learning Representations (ICLR), 2023.
URL https://openreview.net/forum?id=1PL1NIMMrw

[19] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, H. Hajishirzi, P. Clark, Self-refine: Iterative refinement with self-feedback, in: Advances in Neural Information Processing Systems, Vol. 36, 2023.
URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html

[20] Y. Liu, T. Han, S. Ma, J. Zhang, Y. Yang, J. Tian, H. He, A. Li, M. He, Z. Liu, Z. Wu, L. Zhu, D. Li, N. Qiang, D. Shen, T. Liu, B. Ge, Summary of chatgpt-related research and perspective towards the future of large language models, Meta-Radiology 1 (2) (2023) 100017. doi:10.1016/j.metrad.2023.100017.

[21] X. Yue, B. Wang, K. Li, Z. Su, H. Sun, Automatic evaluation of attribution by large language models, in: Findings of the Association for Computational Linguistics: EMNLP 2023, 2023, pp. 4615–4635. doi:10.18653/v1/2023.findings-emnlp.307.

[22] S. K. Mani, K. Hsieh, S. Segarra, R. Chandra, Y. Zhou, S. Kandula, Securing public cloud networks with efficient role-based micro-segmentation, in: Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association, 2025, pp. 1033–1048.
URL https://www.usenix.org/conference/nsdi25/presentation/mani

[23] D. Canavese, R. Laborde, A. Laraba, A. Ferreira, A. Benzekri, Game of zones: An automated intent-based network micro-segmentation methodology, in: Proceedings of the 2025 IEEE/IFIP Network Operations and Management Symposium (NOMS), IEEE, 2025, pp. 1–9. doi:10.1109/NOMS57970.2025.11073630.

[24] C. Katsis, E. Bertino, ZT-SDN: An ML-powered zero-trust architecture for software-defined networks, ACM Transactions on Privacy and Security 28 (2) (2025). doi:10.1145/3712262.

[25] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, M. Chiesa, NetConfEval: Can LLMs facilitate network configuration?, Proceedings of the ACM on Networking 2 (CoNEXT2) (2024) 1–25. doi:10.1145/3656296.

[26] A. Angi, A. Sacco, G. Marchetto, LLNet: An intent-driven approach to instructing softwarized network devices using a small language model, IEEE Transactions on Network and Service Management 22 (4) (2025) 3403–3418. doi:10.1109/TNSM.2025.3570017.

[27] R. Mondal, N. Bjørner, T. Millstein, A. Tang, G. Varghese, LLM-based config synthesis requires disambiguation (2025). arXiv:2507.12443.

[28] L. Mhamdi, M. M. Isa, Securing SDN: Hybrid autoencoder-random forest for intrusion detection and attack mitigation, Journal of Network and Computer Applications 225 (2024) 103868. doi:10.1016/j.jnca.2024.103868.

[29] L. D. Manocchio, Y. Chen, S. Layeghy, D. Gwynne, M. Portmann, P4-Secure: In-band DDoS detection in software defined networks, IEEE Transactions on Network and Service Management 22 (2) (2025) 2120–2137. doi:10.1109/TNSM.2025.3552844.

[30] Y. Bello, A. R. Hussein, Dynamic policy decision/enforcement security zoning through stochastic games and meta learning, IEEE Transactions on Network and Service Management 22 (1) (2025) 807–821. doi:10.1109/TNSM.2024.3481662.

[31] N. N. Hari, P. Krishnan, K. Jain, A. K. J. Saudagar, P. Prabu, R. C. Poonia, A three-tier microsegmentation framework for enterprise networks under zero trust architecture, Alexandria Engineering Journal 141 (2026) 150–164. doi:10.1016/j.aej.2026.03.014.

[32] S. G. E. Gökstorp, J. Nyberg, Y. Kim, P. Johnson, G. Dán, Anomaly detection in security logs using sequence modeling, in: Proceedings of the 2024 IEEE/IFIP Network Operations and Management Symposium (NOMS), IEEE, 2024, pp. 1–9. doi:10.1109/NOMS59830.2024.10575561.

[33] H. V. Vo, H. N. Nguyen, T. N. Nguyen, H. P. Du, SDAID: Towards a hybrid signature and deep analysis-based intrusion detection method, in: Proceedings of the 2022 IEEE Global Communications Conference (GLOBECOM), IEEE, 2022, pp. 2615–2620. doi:10.1109/GLOBECOM48099.2022.10001582.

[34] H. V. Vo, H. P. Du, H. N. Nguyen, AI-powered intrusion detection in large-scale traffic networks based on flow sensing strategy and parallel deep analysis, Journal of Network and Computer Applications 220 (2023) 103735. doi:10.1016/j.jnca.2023.103735.

[35] K. Dzeparoska, J. Lin, A. Tizghadam, A. Leon-Garcia, LLM-based policy generation for intent-based management of applications, in: Proceedings of the 2023 19th International Conference on Network and Service Management (CNSM), IEEE, 2023, pp. 1–7. doi:10.23919/CNSM59352.2023.10327837.

[36] M. Siino, F. Giuliano, I. Tinnirello, Integrating large language models into network testbeds: A novel approach for automated experimentation and optimization, Journal of Network and Computer Applications 243 (2025) 104281. doi:10.1016/j.jnca.2025.104281.

[37] K. Yamamoto, M. Katsurai, Verification-guided specification synthesis with large language models for intrusion detection rules (2026). arXiv:2608.22889.

[38] P. Cichonski, T. Millar, T. Grance, K. Scarfone, Computer security incident handling guide, NIST Special Publication 800-61 Rev. 2, National Institute of Standards and Technology (2012). doi:10.6028/NIST.SP.800-61r2.

[39] A. Nelson, M. Souppaya, K. Scarfone, Incident response recommendations and considerations for cybersecurity risk management, NIST Special Publication 800-61 Rev. 3, National Institute of Standards and Technology (2025). doi:10.6028/NIST.SP.800-61r3.

[40] I. Belhaj, Q. Zhang, C. Alcaraz, J. Lopez, Cybersecurity automation and orchestration: A survey, ACM Computing Surveys 56 (2) (2023) 1–36. doi:10.1145/3605776.

[41] H. P. Du, M. Q. Tran, T. T. Nguyen, H. N. Nguyen, Multi-layer defense for AI-powered IDS: Ensemble adversarial training and explainable resilience to evasion attacks, in: Proceedings of the 2025 2nd International Conference on Cryptography and Information Security (VCRIS), IEEE, 2025, pp. 1–6. doi:10.1109/VCRIS68011.2025.11250565.

[42] H. V. Vo, H. P. Du, H. N. Nguyen, MDOB: Enhancing resilient and explainable AI-powered malware detection using feature set optimization and mutual deep+boosting ensemble inference, Journal of Information Security and Applications 93 (2025) 104175. doi:10.1016/j.jisa.2025.104175.
URL https://www.sciencedirect.com/science/article/pii/S2214212625002121

[43] H. V. Le, T. N. Nguyen, H. N. Nguyen, L. Le, An efficient hybrid webshell detection method for webserver of marine transportation systems, IEEE Transactions on Intelligent Transportation Systems 24 (2) (2023) 2630–2642. doi:10.1109/TITS.2021.3122979.

[44] R. Doriguzzi-Corin, L. A. D. Knob, L. Mendozzi, D. Siracusa, M. Savi, Introducing packet-level analysis in programmable data planes to advance network intrusion detection, Computer Networks 239 (2024) 110162. doi:10.1016/j.comnet.2023.110162.

[45] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, J. Turner, OpenFlow: Enabling innovation in campus networks, in: ACM SIGCOMM Computer Communication Review, Vol. 38, 2008, pp. 69–74. doi:10.1145/1355734.1355746.

[46] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, D. Walker, P4: Programming protocol-independent packet processors, ACM SIGCOMM Computer Communication Review 44 (3) (2014) 87–95. doi:10.1145/2656877.2656890.

[47] H. Farhady, H. Lee, A. Nakao, Software-defined networking: A survey, Computer Networks 81 (2015) 79–95. doi:10.1016/j.comnet.2015.02.014.

[48] C. Jung, M. Ghasemi, T. Benson, J. Rexford, A scalable and dynamic ACL system for in-network defense, in: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, ACM, 2022, pp. 231–245. doi:10.1145/3548606.3560606.

[49] Y.-D. Lin, H.-X. Huang, D. Sudyana, Y.-C. Lai, AI for AI-based intrusion detection as a service: Reinforcement learning to configure models, tasks, and capacities, Journal of Network and Computer Applications 229 (2024) 103936. doi:10.1016/j.jnca.2024.103936.

[50] H.-L. Huynh, Q.-C. Tang, V.-T. Phan, K. Nguyen-An, Design and evaluation of a controlled post-alert incident orchestration and response subsystem using a rule engine and a local large language model (2026). arXiv:2609.26316.

[51] S. Barbieri, L. V. de Meneses, Á. L. R. Ferraz, L. A. Pereira Júnior, SOCpilot: Verifying policy compliance for LLM-assisted incident response (2026). arXiv:2605.05501.

[52] M. J. Zellinger, R. Liu, M. Thomson, Cost-saving LLM cascades with early abstention (2025). arXiv:2502.09054.

[53] F. Metzger, R. Irmer, I. Grønbæk, K. Tutschku, Intent-based networking: Concepts and definitions, in: Proceedings of the 2023 IEEE Conference on Network Softwarization (NetSoft), IEEE, 2023, pp. 1–7. doi:10.1109/NetSoft57336.2023.10175424.

[54] D. D. A. Nguyen, F. Autrel, A. Bouabdallah, G. Doyen, A robust approach for the detection and prevention of conflicts in I2NSF security policies, in: Proceedings of the 2023 IEEE/IFIP Network Operations and Management Symposium (NOMS), IEEE, 2023, pp. 1–7. doi:10.1109/NOMS56928.2023.10154304.

[55] D. Bringhenti, G. Marchetto, R. Sisto, F. Valenza, J. Yusupov, Automated firewall configuration in virtual networks, IEEE Transactions on Dependable and Secure Computing 20 (2) (2023) 1559–1576. doi:10.1109/TDSC.2022.3160293.

[56] W. Zahwa, A. Lahmadi, M. Rusinowitch, M. Ayadi, Deep reinforcement learning for in-network placement of ACL rules under constraints, in: Proceedings of the 2025 21st International Conference on Network and Service Management (CNSM), IEEE, 2025, pp. 1–7. doi:10.23919/CNSM67658.2025.11297518.

[57] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, C. B. Thomas, MITRE ATT&CK: Design and philosophy, Tech. Rep. MP180360, The MITRE Corporation (2018).
URL https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf

[58] MITRE Corporation, MITRE ATT&CK: Enterprise matrix, https://attack.mitre.org/, accessed: 2026-06-04 (2026).

[59] Linux Foundation, SONiC: Software for open networking in the cloud, https://sonicfoundation.dev/, accessed: 2026-06-04 (2026).

Algorithm 1 Fail-closed bounded-isolation decision

Input: alert $a = (s, d, \rho, \pi, \sigma, \ell, \kappa, \tau, t)$; segmentation policy P; baseline model B; protected set $\mathcal{N}$; active rules $\mathcal{R}$; compiled program Π (if active); confidence threshold τ_c; TTL bounds $[\delta_{\min}, \delta_{\max}]$; blast-radius limits $\mathcal{L}$

Output: an authorized, provenance-marked bounded DROP rule, or a non-enforcing decision NONE/LOG_ONLY

$a' \leftarrow$ NormalizeAndDeduplicate(a)
if a' is invalid or duplicated within the suppression window **then**
    $o \leftarrow$ NONE
else if $\ell \notin \{\text{P1}, \text{P2}\}$ **then**
    $o \leftarrow$ LOG_ONLY
else
    $d \leftarrow \Pi(\text{Features}(a', \text{History}(s)))$ ▷ compiled path
    if Π is inactive or $d.\alpha =$ ABSTAIN **then** ▷ runtime path
        $ctx \leftarrow$ RetrieveContext($a', P, B, \mathcal{R}$)
        $ctx \leftarrow ctx \cup$ Neighbors(s) $\cup$ SimilarIncidents(a') $\cup$ BlockImpact(s)
        if CacheHit(a') **then**
            $d \leftarrow$ CachedDecision(a')
        else
            $d \leftarrow$ LLMDecision(a', ctx)
        end if
    end if
    if d fails schema validation **then**
        $o \leftarrow$ NONE
    else if $d.\alpha \neq$ DROP **then**
        $o \leftarrow$ LOG_ONLY
    else if $d.\theta \neq s$ or $d.\theta \in \mathcal{N}$ **then**
        $o \leftarrow$ NONE
    else if ConflictOrDuplicate($d, \mathcal{R}$) **then**
        $o \leftarrow$ NONE
    else if $d.c < \tau_c$ **then**
        $o \leftarrow$ LOG_ONLY
    else if BlastRadiusExceeded($d, \mathcal{L}$) or CircuitBreakerOpen() **then**
        $o \leftarrow$ NONE
    else
        $d.\delta \leftarrow$ Clamp($d.\delta, \delta_{\min}, \delta_{\max}$)
        $r \leftarrow$ GenerateDropIntent(d, agent)
        $o \leftarrow$ AuthorizeAndEnforce(r)
    end if
end if
RecordAudit(a', d, o)